CAWASMA

Context-Aware Web & API Security Misconfiguration Analyzer

Final-year project — viva study notes

Atta Ur Rehman

Backend & scanner engine

Roll number: 22P-9236

Supervisor: Dr. Bilal Khan

Team: Asim Shakeel (22P-9229)
Neelam Jabeen (22P-9353)
Atta Ur Rehman (22P-9236)

These notes are generated from the repository, documentation and git history.
Every number is cited to the file or command it came from. Date: November 10, 2026.

Contents

1 The whole project **2**
- 1.1 The problem . . . 2
- 1.2 The idea . . . 2
- 1.3 Architecture . . . 2
- 1.4 Technology, and why . . . 3
- 1.5 Team roles and the shared contracts . . . 3

2 What Atta built **4**
- 2.1 HTTP engine and the SSRF guard . . . 4
- 2.2 Crawler . . . 5
- 2.3 Check framework and detection functions . . . 5
- 2.4 CVSS scoring code . . . 5
- 2.5 Attack-chain engine . . . 6
- 2.6 Pipeline orchestration . . . 6
- 2.7 FastAPI application and the API . . . 6
- 2.8 Celery workers . . . 7
- 2.9 Database and migrations . . . 7
- 2.10 Reports . . . 7

3 Numbers to know **7**
- 3.1 Detection vs. ZAP and Nuclei . . . 7
- 3.2 Coverage, scoring split and tests . . . 8
- 3.3 Ranking result, honestly . . . 8

4 Contribution timeline **8**

5 Viva preparation **9**
- 5.1 Likely questions, with short answers . . . 9
- 5.2 Three-minute demo script . . . 10
- 5.3 Five things to explain without notes . . . 10

6 Glossary **10**

1 The whole project

1.1 The problem

A normal security scanner gives a fixed severity to each kind of finding. A missing `X-Content-Type-Options` header is "medium" wherever it is found: on a public brochure page, and on a login API that returns credentials. The person reading the report then has to work out, by hand, which of a hundred "medium" findings actually matters. That triage is slow and easy to get wrong. Tools such as OWASP ZAP and Nuclei do find real issues, but they rate the issue, not the place it sits in.[1]

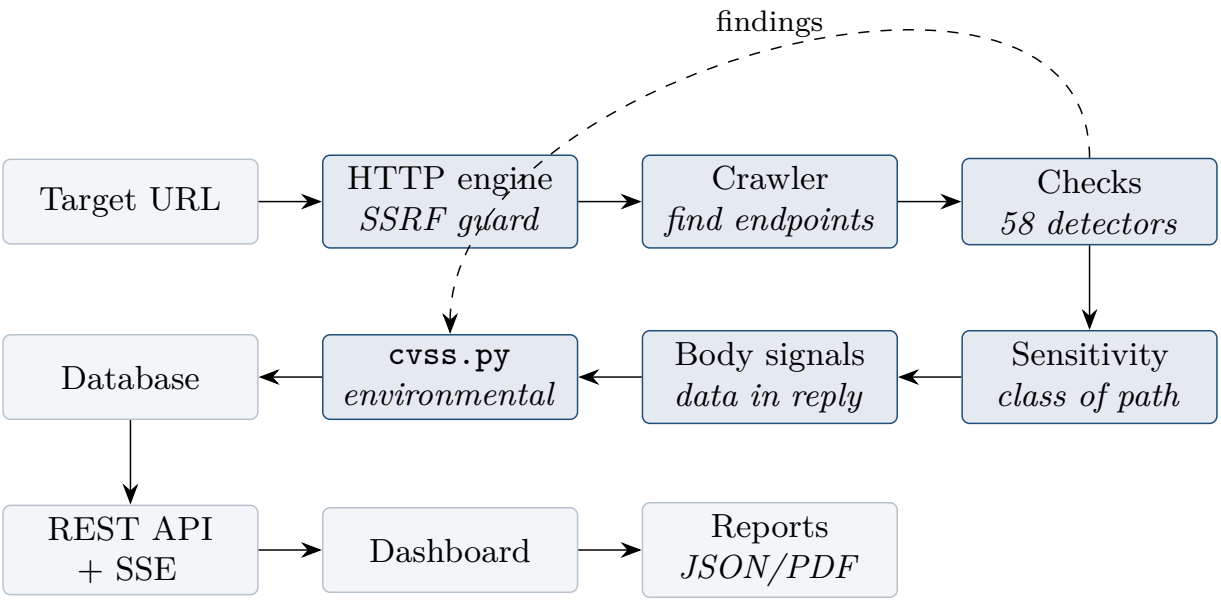
1.2 The idea

CAWASMA scores the *same* finding differently according to where it occurs and what the endpoint returned. It does this with the standard CVSS v3.1 *environmental* metrics, so the result is not an invented number: it is a recomputation of the official score with a vector anyone can check on the FIRST calculator.

One-paragraph pitch. CAWASMA is a defensive, black-box, HTTPS-only scanner for web apps and REST APIs. Each of its checks carries a fixed CVSS v3.1 base vector. For every finding it works out how sensitive the endpoint is (from the URL path) and what sensitive data the response holds (credentials, personal data, financial data, internal IPs). It turns those into CVSS environmental security requirements (CR/IR/AR) and recomputes the score, so a login API and a public page that share a weakness no longer share a priority. The point of the project is *prioritisation, not detection.*

1.3 Architecture

A scan is a pipeline. A URL goes in; a ranked, explained report comes out.



The stages are: the **engine** fetches safely (one SSRF check per redirect hop); the **crawler** discovers endpoints (seed, HTML, JavaScript, a wordlist, and OpenAPI paths); the **checks** look for misconfigurations; the **sensitivity** classifier labels each endpoint CRITICAL/HIGH/MEDIUM/LOW

[1] Source: `docs/supervisor/scanners.md`

from its path; the **body classifier** looks for sensitive data in the reply; `cvss.py` turns those two into CR/IR/AR and recomputes the score; the result is stored, served by the API over REST and Server-Sent Events, shown on the dashboard, and exported as reports.

1.4 Technology, and why

Part	Choice and reason
Scanner core	Python 3.12 with `httpx` + `asyncio`: endpoints are analysed concurrently, and the core stays database-free so it is easy to test.
Scoring	The `cvss` library, so the environmental maths is the published CVSS v3.1 formula, not ours.
NLP	Sentence-Transformers (`all-MiniLM-L6-v2`) and spaCy for the sensitivity class; the engine still works with the built-in classifier when these are absent.
API	FastAPI, with Pydantic schemas and a frozen contract; Celery + Redis run scans in the background with live status.
Database	PostgreSQL in production, SQLite in tests; the models stay portable with a `JSONType` column and database-level cascades.
TLS	SSLyze (with a plain-`ssl` fallback) for the transport checks.
Reports	One HTML template through WeasyPrint for the PDF, so HTML and PDF never drift.
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, shadcn/ui, TanStack Query.
Tests	`pytest` (backend), Vitest + Testing Library and Playwright (frontend).

1.5 Team roles and the shared contracts

Three people own three areas, and they meet at three written contracts, frozen so one person's change cannot silently break another's code.[2]

Member	Owns
Asim (22P-9229)	Check specifications and their CVSS v3.1 vectors, the environmental mapping, chain templates, the sensitivity and response (ML/NLP) classifiers, and the whole evaluation lab.
Neelam (22P-9353)	The Next.js web app, the `frontend` service and its CI job.
Atta (22P-9236)	The backend: HTTP engine, crawler, check functions, CVSS scoring code, attack-chain engine, reports, FastAPI app, Celery workers, database and migrations, plus the shared infrastructure and reviews.

API contract (v1). `app/api/schemas/contract.py` and `docs/api-contract.md`: the field names the frontend reads (`check_id`, `base_score`, `final_score`, status `DONE`).

ML / CVSS contract. `app/core/contracts.py` and `docs/cvss-mapping-contract.md`: the types the engine and Asim's classifiers exchange, so the engine keeps working with the built-in classifiers when the ML packages are missing.

Check-spec format. `app/core/specs.py` and `docs/check-spec-format.md`: how a check is declared so the registry can run it.

Two scoring states. Not every check is re-scored the new way yet. Of 62 check ids, 25 have an approved specification and are scored through the CVSS v3.1 environmental metrics; the other 37 still use an interim formula, $\min(10,\ \text{base} \times \text{context multiplier} + \text{response bonus})$, and each finding records which method scored it (`scoring_method`).[3]

[2] Source: `CLAUDE.md; .github/CODEOWNERS`

[3] Source: `make coverage-stats; docs/06-scoring-model.md`

2 What Atta built

Atta owns the backend: everything that turns a URL into stored, scored, served findings. The work is in `backend/app/core/` (the database-free scanner logic), `backend/app/api/` (FastAPI), `backend/app/workers/` (Celery), `backend/app/db/` and `backend/migrations/`, plus the shared infrastructure (`docker-compose.yml`, `Makefile`, `.github/`).[4]

2.1 HTTP engine and the SSRF guard

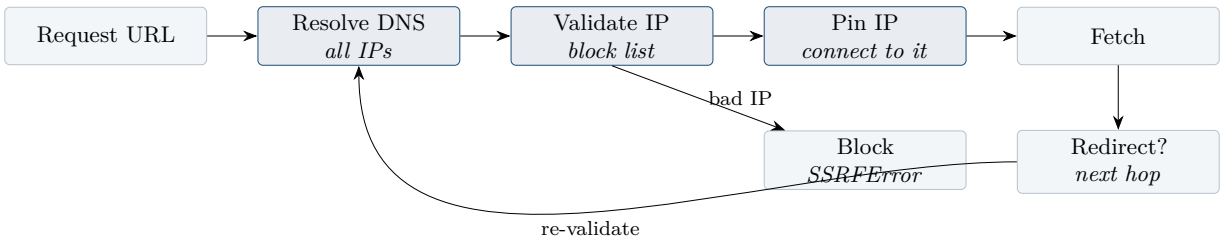
What it does. The engine (`core/engine.py`) makes every network request for a scan. It keeps the scan inside one host, follows redirects safely, and refuses to touch private or internal addresses. The safety logic lives in `utils/security.py`.

How it works.

- `HttpEngine(scope_host=...)` fetches with `httpx` and `asyncio`. Plain GET/HEAD are cached per scan by `fetch()`, so the same URL is not fetched twice.
- Before each connection, `resolve_and_validate()` resolves the host and checks *every* resolved IP with `_ip_is_disallowed()`. A redirect is a new hop, so the check runs again for the next host.
- `PinnedNetworkBackend` connects to the exact IP that was validated, so DNS cannot change between the check and the connection (DNS rebinding).
- The engine keeps no cookies between requests and records `Set-Cookie` metadata (never values) on every hop. In passive mode it sends only GET/HEAD.

Listing 1: SSRF block list on the resolved IP (`utils/security.py`).

```python
def _ip_is_disallowed(ip, *, internal_ok=False):
    inner = _embedded_ipv4(ip) # unwrap ::ffff:a.b.c.d
    if inner is not None:
        r = _ip_is_disallowed(inner, internal_ok=internal_ok)
        return f"{r} (via {ip})" if r else None
    if ip in METADATA_IPS: return "cloud metadata address"
    if ip.is_loopback: return "loopback address"
    if ip.is_link_local: return "link-local address"
    if ip.is_reserved: return "reserved address"
    if internal_ok: return None # tests / eval only
    if ip.is_private: return "private address"
    return None
```



Design decisions. Validating the *resolved IP* (not the hostname) and pinning it is what stops DNS rebinding; a hostname check alone would not. `allow_private` exists only for tests and the evaluation harness, where the targets are on a private Docker network; in normal runs `ALLOWED_INTERNAL_HOSTS` is empty.

[4] Source: `.github/CODEOWNERS; CLAUDE.md`

Inputs/outputs. In: a URL and the scan scope. Out: a validated `ScanResponse` used by the crawler and the checks. **Tested by** `tests/security/` (loopback, link-local, metadata, IPv4-mapped IPv6, and rebinding cases) and `tests/unit` engine tests.

Limitations. The guard blocks by IP class; a public host that an attacker controls and that itself proxies inward is out of scope, as is any target that requires a login.

2.2 Crawler

What it does. `core/crawler.py` discovers endpoints to scan from a single seed URL.

How it works. It reads the seed page, follows in-scope HTML links, extracts routes from JavaScript, tries a wordlist, and reads OpenAPI/Swagger GET paths and `/rest/`, `/vN/` routes. `_state_changing()` blocks anything named like an action (`/createdb`, `/logout`, `/reset`) so the scan never changes server state. A soft-404 baseline stops pages that answer 200-but-empty from being treated as real endpoints.

Design decisions. Discovery is deterministic (fixed order) so a scan is reproducible. Only GET/HEAD are used to discover. **Tested by** crawler unit tests and `tests/unit/test_scan_performance.py` (state-changing route recognition). **Limitation:** no JavaScript execution, so routes that only appear after client-side rendering are missed.

2.3 Check framework and detection functions

What it does. A check is a pure function `detect(ctx: CheckContext) -> RawFinding | None`, registered with `@rule`. Atta writes the detection functions (one module per category in `core/checks/`); Asim owns the catalog entry and CVSS vector.

How it works. The registry (`core/checks/registry.py`) also runs the newer *spec-driven* checks from `config/checks.yaml` with a `SpecContext`. A guard test fails if a check has no fire/no-fire test. There are 58 catalog checks; `checks.yaml` adds 4 spec-only ids, 62 in all.[5]

Design decision. Keeping `core/` database-free means a check reads only its `CheckContext` and can be unit-tested with no server. **Tested by** `tests/checks/` (a fire and a no-fire test per check).

2.4 CVSS scoring code

What it does. `core/cvss.py` is the scoring engine. It computes the environmental (context-aware) score from a base vector plus the endpoint class and body signals, and it keeps the interim legacy formula for checks that do not yet have a spec.

How it works. `environmental_score()` builds the full CVSS vector by appending CR/IR/AR, then uses the `cvss` library to compute both base and environmental scores, so the maths is the published formula. The requirements themselves come from Asim's mapping via `security_requirements()`. `score_finding()` looks up the check's base vector from the spec registry.

Listing 2: Append CR/IR/AR, recompute with the `cvss` library (`core/cvss.py`).

```
reqs, reasons = security_requirements(sensitivity, signals, mapping)
tail = "".join(f"/{m}:{reqs[m]}" for m in ("CR", "IR", "AR"))
scored = CVSS3(base_vector + tail)
base, final = scored.base_score, scored.environmental_score
if all(v == "X" for v in reqs.values()):
    reasons.append("no-context-change") # public: base stands
```

[5] Source: `make check-coverage; backend/app/core/checks/`

Why not our own formula? Because then the number would be ours to defend. By appending CR/IR/AR to the official base vector and letting the `cvss` library compute it, the final score is a standard CVSS v3.1 environmental score that anyone can reproduce on the FIRST calculator. Our only claim is the *mapping* from context to CR/IR/AR.

Inputs/outputs. In: a check's base vector, the sensitivity class, the body signals, and Asim's mapping. Out: a `ContextScore` (base, final, severity bucket, the full vector, and the `score_reasons`) stored on the finding. **Tested by** `tests/unit/test_cvss.py`. **Limitation:** the 37 legacy checks are not on this path yet; they record `scoring_method = legacy`.

2.5 Attack-chain engine

What it does. `core/chains.py` correlates several findings into one attack chain and scores it with a composite vector.

How it works. `correlate()` reads the templates from `config/chains.yaml` (Asim's file), groups findings by scope unit, and emits a `DetectedChain` per matched template. Each member's evidence names the chain. **Tested by** `tests/unit/test_chains.py`. **Limitation:** chains only form where a template matches; on the four evaluation targets they appear only on DVWA and crAPI.

2.6 Pipeline orchestration

What it does. `core/pipeline.py` is the conductor: `run_pipeline()` drives crawl → checks → classify → score → chains and produces a `ScanResult`.

How it works. It analyses endpoints concurrently with a bounded limit (`MAX_CONCURRENT_ENDPOINTS`) in a fixed order, so a scan is both fast and reproducible. `_analyse_endpoint()` runs the checks for one endpoint, asks for its sensitivity class and body signals, and scores each finding. `StageClock` records per-stage timings into `stats[stage_seconds]`. Progress is written to `Scan.progress_detail` after each step, and `cancel_requested` is checked there too. An unreachable target raises `ScanError` — the only error text shown to users.

Design decision. The pipeline is the one place that depends on both the engine and the classifiers, so it holds the fallback logic (`_ml_classifier()`): if the ML classifier is absent it uses the built-in one, honouring the contract. **Tested by** `tests/integration/` (full scans against fixtures). **Limitation:** a very large site is bounded by `SCAN_TIMEOUT` rather than scanned exhaustively.

2.7 FastAPI application and the API

What it does. `api/` exposes the scanner over REST and Server-Sent Events. `POST /api/scan` starts a scan; other routes read results, a single finding, history, and exports.

How it works.

- `services/scan_dispatch.py` queues the scan on Celery with the task id equal to the scan id, returning 503 `QUEUE_UNAVAILABLE` if the broker is down. `RUN_SCANS_INLINE` runs it as a FastAPI background task instead.
- Live status is SSE at `GET /api/scan/{id}/events`, which reads the database (at most `SSE_MAX_STREAMS_PER_CLIENT` open per client).
- Cancel sets `Scan.cancel_requested`. Scanning is rate-limited with a Redis fixed window (429; `api/limits.py`).
- Errors use the envelope `{success:false, error:{code,message}}`. Routes return Pydantic schemas from `api/schemas/`, never ORM objects.

Design decision. The contract field names (`check_id`, `base_score`, `final_score`) are mapped in `api/schemas/` rather than renaming database columns, so the database and the API can evolve separately. **Tested by** `tests/api/` (contract schemas, the OpenAPI export, and the

documented examples). **Limitation:** SSE needs a database read loop; there is no push from the worker, so status lags by at most one poll interval.

2.8 Celery workers

What it does. `workers/` runs scans off the request thread. `cawasma.run_scan` executes the pipeline and writes results.

How it works. At start-up the API marks any `api`-runner scan still QUEUED/RUNNING as FAILED (it died with the old process); Celery scans carry `options.runner = "celery"`. The task id is the scan id, so status and cancel line up. **Tested by** worker and dispatch tests. **Limitation:** a worker crash mid-scan leaves a scan to be reaped at the next start-up rather than resumed.

2.9 Database and migrations

What it does. `db/` holds the SQLAlchemy models and session; `migrations/` holds the Alembic history (0001–0005).

How it works. Models stay portable so the same code runs on SQLite (in tests) and PostgreSQL (in production): JSON columns use `JSONType` (JSONB on PostgreSQL), and deletes rely on database `ON DELETE` cascades, which are enforced on SQLite too. The official base score and the context-aware score are separate columns on the `findings` table.

Design decision. Schema changes go through `alembic revision -autogenerate`; a merged migration is never edited and tables are never created by hand, so every environment has the same schema. **Tested by** `tests/db/` and the CI Alembic upgrade/downgrade. **Limitation:** keeping two databases portable rules out some PostgreSQL-only features.

2.10 Reports

What it does. `core/reports.py` renders a scan as JSON, CSV, HTML or PDF, with no database access (it takes an assembled data dictionary from `services/report_service.py`).

How it works. `score_story()` turns a finding's `score_reasons` into the plain-English "why this score" lines; HTML and PDF both come from `app/templates/report.html.j2`, the PDF being that same HTML through WeasyPrint, so the two can never drift. `redact_evidence()` runs again on every export.

Design decision. Keeping the renderers database-free means a report can be produced and unit-tested from a fixture. **Tested by** report unit tests. **Limitation:** the PDF depends on WeasyPrint's CSS support, so the template avoids layout features it renders poorly.

3 Numbers to know

All figures are from the repository at commit `7cc819f` on `main`.

3.1 Detection vs. ZAP and Nuclei

Four deliberately vulnerable targets, each tool scored only on the checks its rules map to.[6]

Tool	scope	TP	FP	FN	Precision	Recall	Time (s)
CAWASMA	all (micro)	25	0	8	1.00	0.758	10.5
ZAP (baseline)	all (micro)	7	0	14	1.00	0.333	104.4
Nuclei	all (micro)	13	0	4	1.00	0.765	126.2

Per-target CAWASMA recall: Juice Shop 0.44, DVWA 1.00, VAmPI 0.75, crAPI 0.83. Read these carefully: ZAP ran passively (no attacks), and the ground truth uses CAWASMA's check ids, which favours CAWASMA's scope.

[6] Source: `eval/results/metrics.csv; eval/results/final/tools.md`

3.2 Coverage, scoring split and tests

Metric	Value
Check ids (catalog 58 + 4 spec-only)	62
With an approved spec / scored environmental	25
Scored by the legacy interim formula	37
Latest eval findings (4 targets)	1230
environmental / legacy	765 (62%) / 465 (38%)
Backend tests (pytest)	978
Frontend unit test files / e2e specs	25 / 7

Source: `make coverage-stats`; pytest collection; `git ls-files`.

3.3 Ranking result, honestly

NDCG@10 on environmentally-scored findings, base vs. context, mean over 1000 tie-breaks (seed 13).[7]

Target	base	context
Juice Shop	0.001	0.000
DVWA	0.147	0.261
VAmPI	0.607	0.591
crAPI	0.539	0.535
Mean	**0.324**	**0.347**

What this means. On four targets the context-aware ranking beats the base score only on DVWA and is otherwise equal or slightly worse. So the evaluation does *not* show the ranking is better overall. The graded pools are not finished, so there is no pool-based NDCG yet. The honest claim the project proves is the *mechanism*: the same weakness on a sensitive, credential-returning endpoint is scored higher than on a public page, with a reproducible CVSS vector.

4 Contribution timeline

Grouped from merged pull requests.[8]

Merged	PRs	What changed
May–24 Oct	#1–#5, #7	Project scaffold, database layer, API contracts.
25 Oct	#13–#16	ML stubs where the contract expects them; data-model and contract gaps.
25 Oct	#22, #23	Async engine and crawler; hardened SSRF guard, `ALLOWED_INTERNAL_HOSTS`.
25–26 Oct	#24, #25	Check registry; CVSS v3.1 environmental scoring from sensitivity + body signals.
26 Oct	#26, #27	Vertical slice; stop tracking the local database.
26–27 Oct	#36, #39	Whole-code bug sweep; full check set.
4 Nov	#45	Celery scans, live SSE status, cancel, rate limit.
4–5 Nov	#52, #53, #57	Final audit (no code changed in phase 1).
5 Nov	#65, #67	Fix `validate-config` crash and the PostgreSQL CI job.
5–9 Nov	#68, #74, #76–#79	CODEOWNERS; `make up` runs the whole app; supervisor docs; docs cleanup.

[7] Source: `eval/results/final/ranking_environmental.md`
[8] Source: `gh pr list (upstream); git log`

Reviews given. Atta also pushed fixes inside teammates' folders on `atta/fix-*` branches for the owner to review: for Asim #56, #58, #60, #61, #62, #70, #71, #72; for Neelam #54, #59, #63, #64, #66, #73, #75.

5 Viva preparation

5.1 Likely questions, with short answers

1. **How do you stop SSRF?** Resolve the host, check every resolved IP against a block list (loopback, link-local, private, reserved, cloud metadata, IPv4-mapped IPv6), then connect to that pinned IP so DNS cannot change. The check re-runs on every redirect hop.
2. **What is DNS rebinding and how do you handle it?** The host resolves to a safe IP at check time and a dangerous one at connect time. `PinnedNetworkBackend` connects to the IP we validated, so the two cannot differ.
3. **Why async httpx?** Endpoints are analysed concurrently with a bounded limit, so scans stay fast (mean 10.5 s per target) without threads.
4. **Why is `core/` database-free?** So a check is a pure function of its `CheckContext` and can be unit-tested with no database or network.
5. **Why not invent your own score?** The final number is a standard CVSS v3.1 environmental score computed by the `cvss` library; only the mapping to CR/IR/AR is ours, and it is auditable.
6. **Base vs. final score — why keep both?** The official base score must stay visible; the context-aware score sits next to it, never replacing it.
7. **The ranking isn't better — so what does the project prove?** It proves the mechanism works and is reproducible: a weakness on a sensitive, credential-bearing endpoint scores higher than on a public page. The ranking study is honest that, on four targets, it is not better overall.
8. **How does a scan run in the background?** `POST /api/scan` queues a Celery task (task id = scan id); progress is written to the database and streamed by SSE; cancel sets a flag checked at each step.
9. **What happens if the broker is down?** The queue endpoint returns 503 `QUEUE_UNAVAILABLE`; `RUN_SCANS_INLINE` can run the scan in the API process instead.
10. **How do you avoid changing the target?** Only GET/HEAD/OPTIONS, and the crawler never requests routes named like actions (`/logout`, `/createdb`).
11. **How is the PDF report produced?** One HTML template through WeasyPrint, so HTML and PDF cannot drift; every export redacts evidence again.
12. **How do you keep secrets out of logs and reports?** `utils/redaction.py` redacts before anything is logged, stored or reported; cookies store metadata, never values.
13. **Why both SQLite and PostgreSQL?** SQLite for fast tests, PostgreSQL in production; models stay portable with `JSONType` and database-level cascades.
14. **How do migrations work?** Alembic autogenerate; a merged migration is never edited; tables are never created by hand.
15. **What is soft-404 handling?** The crawler learns a baseline for "not found" pages that still return 200, so they are not scanned as real endpoints.
16. **Why is XSS/BOLA not covered?** Confirming them needs attack payloads or logged-in accounts, which breaks the safe, read-only, unauthenticated design. BOLA depends on authenticated scanning, which is future work.
17. **How would you scale this?** Scans are already Celery tasks; add workers horizontally. The engine bounds concurrency per scan; the database and Redis are the shared state.
18. **How is a scan cancelled mid-run?** `Scan.cancel_requested` is set and checked at every progress step, so the pipeline stops cleanly.

19. **What is the scope guarantee?** `HttpEngine(scope_host=...)` makes no request to another host and follows no redirect off it.
20. **How are endpoints scored concurrently but deterministically?** Concurrency is bounded and the order is fixed, so results are reproducible.
21. **What does `scoring_method` record?** Whether a finding was scored by the CVSS environmental metrics or by the legacy interim formula.
22. **How do you test the checks?** Every check has a fire and a no-fire test in `tests/checks/`; a guard test fails if one is missing.
23. **What stops a check from being too slow?** Plain GET/HEAD are cached per scan, and per-stage timings are recorded in `stats[stage_seconds]`.
24. **How do you handle an unreachable target?** The pipeline raises `ScanError`, the scan is marked FAILED, and only that message is shown.
25. **Where does the sensitivity class come from in the pipeline?** From `app.ml.sensitivity.classify` (Asim) through the contract, with the built-in classifier as a fallback.
26. **What is the single biggest risk in your code?** The SSRF guard: a gap there would let a scan reach internal services, so it has the most tests.

5.2 Three-minute demo script

1. Start the stack (`make up`) and open the app. Say: "A scan is a Celery task; the API streams live status over SSE."
2. Scan a target. Point at the live progress and the stage timings.
3. Open a finding's detail. Show the base score, the final score, and the CVSS vector with CR/IR/AR appended. Say: "The number is a standard CVSS v3.1 environmental score, not ours."
4. Export the PDF. Say: "Same HTML through WeasyPrint; evidence is redacted again on export."
5. Show the SSRF test file briefly. Say: "Every resolved IP is validated and pinned on every hop."

5.3 Five things to explain without notes

1. The SSRF guard: resolve, validate every IP, pin it, re-check each hop.
2. `environmental_score()`: append CR/IR/AR, recompute with the `cvss` library.
3. Why the base score is always kept next to the final score.
4. The scan lifecycle: queue → Celery → SSE status → cancel flag → stored findings.
5. The honest ranking result and what the project really proves.

6 Glossary

CVSS base score
The standard severity of a vulnerability on its own, 0–10, from a vector such as `AV:N/AC:L/...`. Fixed per check; never invented per scan.

CVSS environmental score
The base score recomputed for a particular environment, using CR/IR/AR and modified base metrics. CAWASMA's context-aware score is this.

CR / IR / AR
Confidentiality, Integrity and Availability *Requirements*: environmental inputs saying how much each property matters here. Values Low, Medium, High, or X (Not Defined). CAWASMA sets them from the endpoint class and the data in the response.

Sensitivity class
CAWASMA's label for how sensitive an endpoint is — CRITICAL, HIGH, MEDIUM or LOW — from the words in its URL path.

Body signals
Sensitive data found in a response: credentials, personal data (PII), financial data, or an internal IP address. Evidence is redacted.

Base vs. final score
CAWASMA always keeps both: the official base score and the context-aware (environmental) final score, in separate columns.

Legacy vs. environmental scoring
A finding is scored either by the CVSS environmental metrics (approved specs) or by the interim legacy formula; its `scoring_method` says which.

Precision
Of the findings that were judged, the fraction that were correct: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$.

Recall
Of the real issues in the answer key, the fraction found: $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

FPR
False-positive rate: $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$.

Ground truth
A hand-written list, per target, of which check should (`present`) or should not (`absent`) fire on which endpoint.

NDCG@10
Normalised Discounted Cumulative Gain at rank 10: a score for how well a ranking puts the important items near the top, against a graded list.

Tie-break sampling
Base scores tie often, so a ranking's NDCG is averaged over 1000 random orderings of the tied items, with a fixed seed.

SSRF
Server-Side Request Forgery: tricking the scanner into requesting an internal address. CAWASMA guards against it by validating the resolved IP on every hop.

Soft-404
A page that returns HTTP 200 but is really "not found". The crawler learns a baseline so it does not treat these as real endpoints.

Passive vs. active scan
Passive sends only safe GET/HEAD and reads what comes back; active sends extra, still non-destructive, probes. CAWASMA never sends attack payloads.

BOLA / IDOR
Broken Object-Level Authorisation / Insecure Direct Object Reference: reading another user's object by changing an id. CAWASMA only flags an *indicator*; confirming it needs logged-in accounts, which is future work.

Attack chain
Several findings that together make a bigger risk, correlated from a template and scored with their own composite vector.

Environmental metric "X"
"Not Defined": the neutral baseline. A public (LOW) endpoint uses X, so its score is unchanged — context only ever raises a score here, it does not lower a public page below its base.